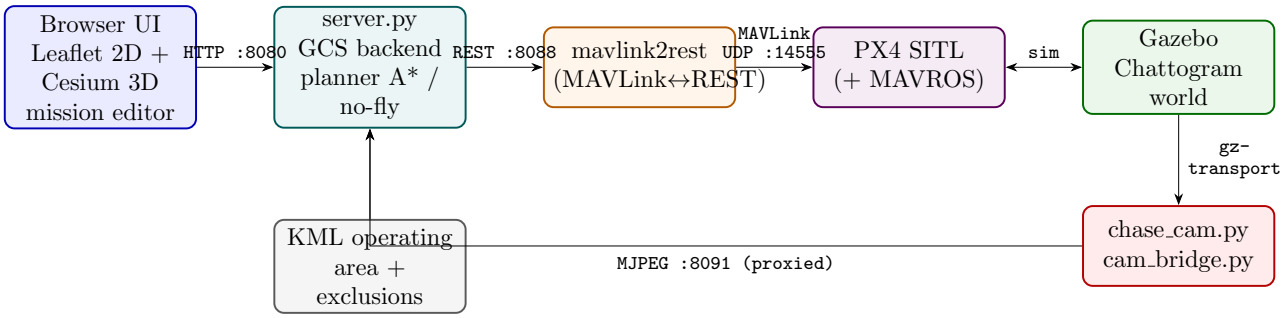
ALBATROSS — System Architecture & Review Response

PX4 SITL + Gazebo (Chattogram) + ROS 2 autonomy + web GCS

August 25, 2026

1 System Architecture

A full software-in-the-loop autonomous maritime-patrol simulator. The autonomy stack builds exclusion-aware waypoints, the flight controller flies them, Gazebo renders the vehicle over the real Chattogram port terrain, and a browser-based Ground Control Station (GCS) provides live ops: map, 3D view, chase-camera feed, and a multi-waypoint mission editor.



1.1 Components

- **PX4 SITL + Gazebo** — flight dynamics + rendering of the x500 over the real Chattogram satellite/terrain world (`PX4_GZ_WORLD=chattogram`).
- **maritime_autonomy** (ROS 2 pkg) — `planner.py` (A* over the KML operating area, exclusion-aware, no-fly enforced), `geo.py`, and mission nodes (`patrol/plan/descend/drop`). MAVROS pushes/arms via the MAVLink mission protocol.
- **mavlink2rest** — exposes MAVLink as REST on `:8088` (`udpin:14555`); telemetry + command bridge for the web GCS. QGroundControl attaches separately on `:14550` (zero-config).
- **maritime_gcs / server.py** (`:8080`) — serves the UI, proxies telemetry, runs the planner, uploads missions, and enforces no-fly zones.
- **chase_cam.py** — spawns a static Gazebo camera that follows the drone (look-at framing) and publishes `/chase_cam`.
- **cam_bridge.py** — encodes the camera image to MJPEG on `:8091` (Wayland/headless-proof; replaces the old x11grab screen-scrape).

1.2 Ports & data flow

8080	GCS web UI + API + /video.mjpeg proxy
8091	chase-cam MJPEG bridge
8088	mavlink2rest REST API
14555	PX4 → mavlink2rest (MAVLink UDP)
14550	PX4 → QGroundControl (MAVLink UDP)

Data path: browser → :8080 server.py → fc_iface → :8088 mavlink2rest → :14555 MAVLink → PX4 SITL ↔ Gazebo. QGroundControl attaches independently on :14550 (PX4's default GCS link). The camera feed flows Gazebo → gz-transport → cam_bridge (:8091) → :8080 proxy → browser.

1.3 Directory layout (consolidated)

Everything lives under one relocatable project root (albatross/):

```
albatross/
  gcs/ web GCS: server.py, fc_iface, chase_cam, cam_bridge, web/
  ros2_ws/ ROS 2 workspace - pkg maritime_autonomy (planner, nodes, world)
  launch/ launch_chattogram_clean.sh, start_all.sh, restart_gcs.sh
  assets/ chattogram_zones.kml (operating area + exclusion zones)
  docs/ this overview + the launch guide (.tex/.pdf)
  media/ demo.mp4 / demo.gif
  external/ symlinks -> PX4-Autopilot and mavlink2rest (kept out of git)
```

Paths are self-locating: server.py derives the root from its own location (override with MARITIME_ROOT) and the shell scripts derive it from their path, so the folder can be renamed or moved freely. PX4-Autopilot and mavlink2rest stay external (multi-GB / third-party) and are symlinked into external/.

2 Review Feedback — Points & Fixability

Point (raised by)	Fix?	How / plan	Effort
Use uXRCE-DDS (microDDS) instead of MAVROS *(Zuhayer)*	**Yes** (planned)	MAVROS was a deliberate sim-first choice (turnkey MAVLink mission protocol: upload WP → AUTO.MISSION → RTL). Move onboard control to px4_msgs over uXRCE-DDS; keep MAVLink only for the GCS link. Caveat: DDS has *no native mission-upload* — the workflow becomes offboard `TrajectorySetpoint` streaming.	Med
ROS 2 node: takeoff + go to a coordinate via DDS *(Zuhayer)*	**Yes**	Offboard node: `OffboardControlMode` + `TrajectorySetpoint` + `VehicleCommand` (ARM, OFFBOARD) over uXRCE-DDS. Good PoC for the DDS path.	Low–Med
Downward-facing camera (nadir) *(Zuhayer M.)*	**Yes** (easy)	Use `gz_x500_mono_cam_down` / add a nadir cam link; bridge as a second web feed next to the chase cam.	Low
AGL determination — which sensor? *(intelis / Akib)*	**Yes**	Real plan: barometer primary + optical rangefinder secondary + GPS. In sim: add a downward distance sensor and fuse via `EKF2_RNG_*` so AGL is validated, not GPS-only.	Low–Med
Flying over water *(intelis)*	Design note	Baro + GPS AGL reliable over open water; optical flow unreliable over featureless water and rangefinder noisy over waves — use baro/GPS over open water, rangefinder near structures/-landing.	Config
Smooth takeoff / landing *(Promit)*	**Done**	PX4 param tuning (below); verified.	Done

Bottom line: every point is addressable. #1 is the only strategic item — nothing is blocked, but DDS will not give mission-upload for free (the one real trade-off vs MAVROS). #2/#3/#4 are quick wins that directly answer the reviewer.

3 Smooth Takeoff & Landing (done)

Tuned PX4 for gentle vertical motion; added to `launch_chattogram_clean.sh` so it applies on every launch.

Param	Value	Effect
MPC_TKO_RAMP_TIME	3.0 s	gradual thrust ramp (no liftoff pop)
MPC_TKO_SPEED	1.0 m/s	gentle climb
MPC_LAND_SPEED	0.4 m/s	soft touchdown
MPC_LAND_ALT1/ALT2	15 / 5 m	slows the descent higher up
MPC_Z_VEL_MAX_DN	1.0 m/s	slower RTL descent
MPC_JERK_AUTO	3.0	jerk-limited, smooth velocity changes
MPC_ACC_UP/DOWN/HOR	2.5 / 2.0 / 2.0	gentle accel transitions

Verified takeoff→hover→land: climb rate stayed within $-0.40 \ldots +0.69$ m/s with no spikes and a soft -0.40 m/s touchdown.